

Operating Systems: Assignment-4

Name: Suraj Shah

Admission Number: I24AI012

Q 1. Perform Following

- a) Obtain a listing of processes by using the ps command.

Command:

ps

Output:

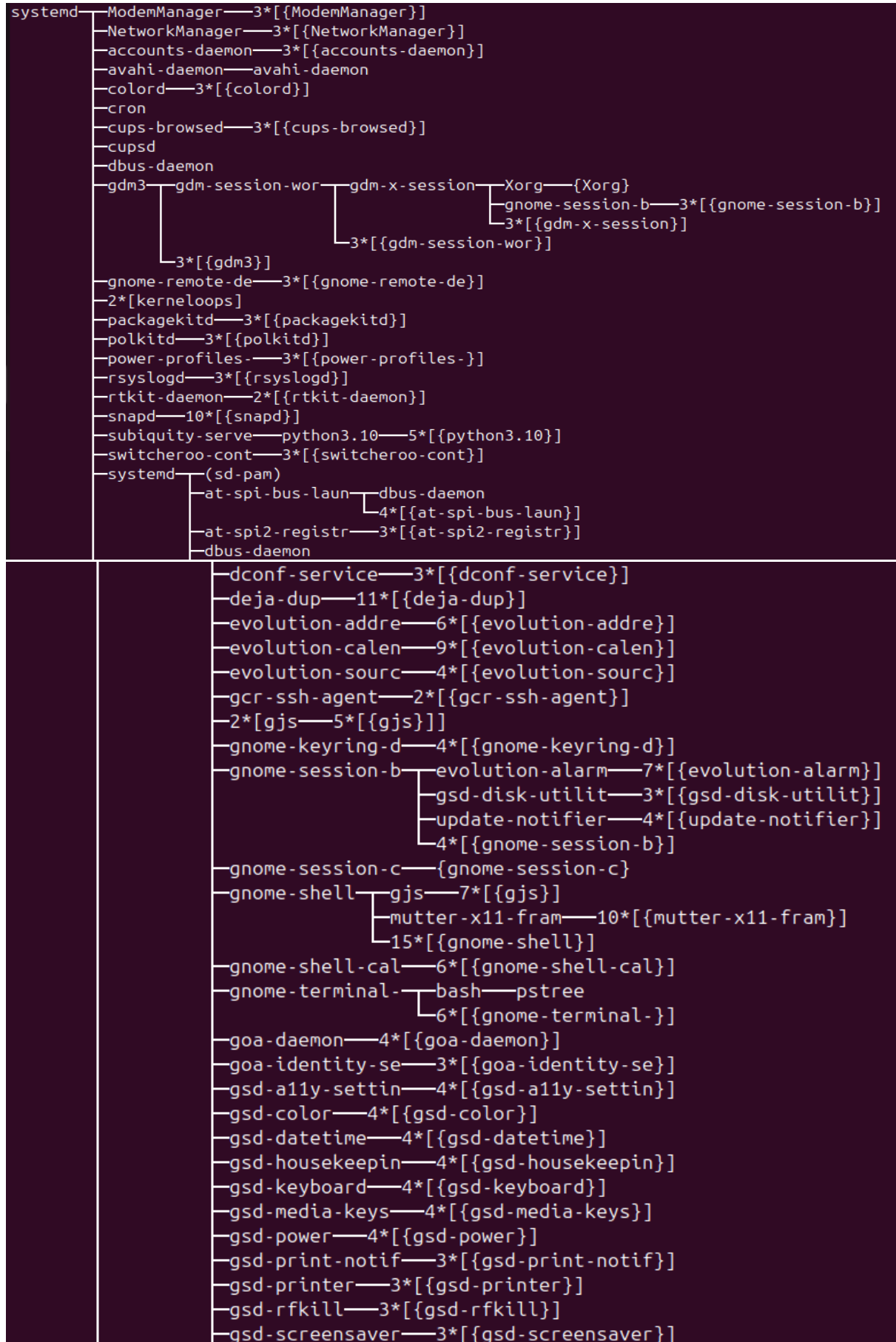
PID	TTY	TIME	CMD
4483	pts/0	00:00:00	bash
4588	pts/0	00:00:00	ps

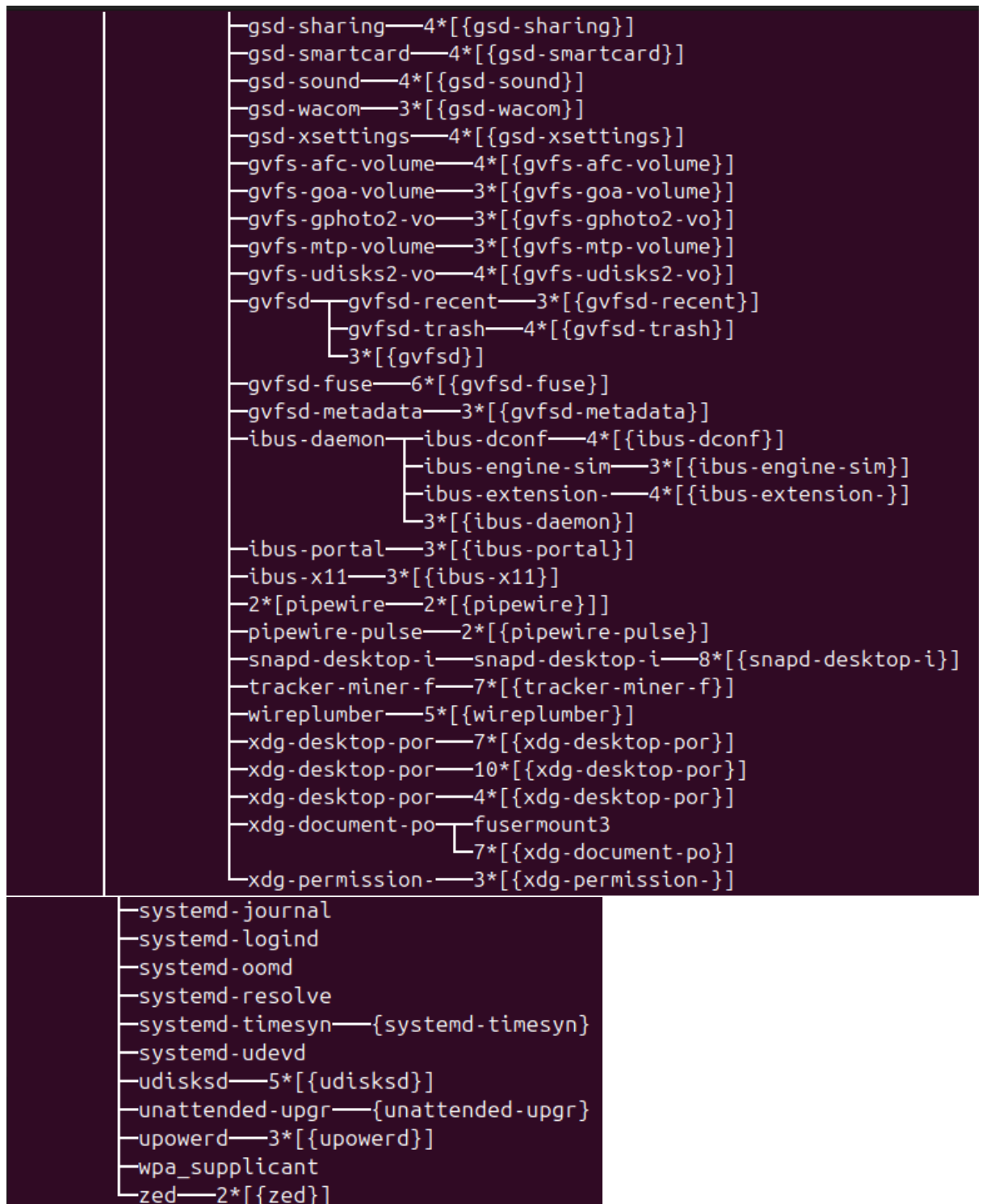
- b) Use a Linux command which displays a tree of all processes in the system.

Command:

ps tree

Output:





c) Use Exec(), Wait() and Exit() system call.

Code:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
  
```

```

int main() {
  
```

```

pid_t pid = fork();

if (pid == 0) {
    printf("Child: Executing ls command\n");
    execl("/bin/ls", "ls", NULL);
    exit(1); // if exec fails
} else {
    wait(NULL);
    printf("Parent: Child finished execution\n");
    exit(0);
}
}

```

Output:

```

ubuntu@ubuntu:~/test$ gcc exec_wait_exit.c -o exec_demo
ubuntu@ubuntu:~/test$ ./exec_demo
Child: Executing ls command
exec_demo  exec_wait_exit.c
Parent: Child finished execution

```

- d) Start a browser and check the process ID of the browser using Top command

Commands:

```

firefox &
top

```

Output:

```

top - 03:41:41 up 21 min, 1 user, load average: 0.70, 0.43, 0.41
Tasks: 229 total, 2 running, 227 sleeping, 0 stopped, 0 zombie
%Cpu(s): 1.4 us, 2.7 sy, 0.0 ni, 95.7 id, 0.0 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 3916.6 total, 152.9 free, 1938.8 used, 2406.9 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1977.8 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2281	ubuntu	20	0	332376	112420	73336	S	2.6	2.8	0:26.39	Xorg
2501	ubuntu	20	0	3963016	387232	151420	S	2.6	9.7	0:41.16	gnome-shell
4994	ubuntu	20	0	3034236	372684	162704	S	2.6	9.3	0:13.29	firefox
5378	ubuntu	20	0	2503384	154728	91080	S	2.3	3.9	0:02.42	Privileged Cont
4471	ubuntu	20	0	850936	54024	42664	S	1.3	1.3	0:07.79	gnome-terminal-
5553	ubuntu	20	0	2473540	120240	82640	S	1.0	3.0	0:00.77	Isolated Web Co
5345	ubuntu	20	0	2452988	97176	66572	S	0.7	2.4	0:00.44	WebExtensions
1285	systemd+	20	0	17560	7812	6916	S	0.3	0.2	0:00.90	systemd-oomd
2724	ubuntu	20	0	468024	7696	7056	S	0.3	0.2	0:00.09	gsd-housekeepin
3119	ubuntu	20	0	354232	29572	18744	S	0.3	0.7	0:01.92	ibus-extension-
4444	ubuntu	20	0	498488	26792	20136	S	0.3	0.7	0:00.31	update-notifier
1	root	20	0	23520	14704	9712	S	0.0	0.4	0:02.71	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.02	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-netns
12	root	20	0	0	0	0	I	0.0	0.0	0:00.01	kworker/u8:0-ipv6_addrconf
13	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mm_percpu_wq
14	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_kthread
15	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_rude_kthread
16	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_trace_kthread
17	root	20	0	0	0	0	S	0.0	0.0	0:01.73	ksoftirqd/0

- e) Kill the browser process using “kill” command

Command:

kill 4994

Output:

```
ubuntu@ubuntu:~/test$ kill 4994
ubuntu@ubuntu:~/test$ Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
```

- f) Change the priority of any user process by using “nice” command

Command:

renice -n 5 -p 10324

Output:

```
top - 04:32:16 up 1:11, 1 user, load average: 0.27, 0.22, 0.13
Tasks: 224 total, 1 running, 223 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.3 us, 0.6 sy, 0.2 ni, 98.6 id, 0.0 wa, 0.0 hi, 0.3 si, 0.0 st
MiB Mem : 3916.6 total, 384.7 free, 1896.6 used, 2284.3 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 2020.0 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2501	ubuntu	20	0	3964112	388388	151516	S	0.9	9.7	1:21.86	gnome-shell
10324	ubuntu	25	5	2928696	331700	153420	S	0.9	8.3	0:08.24	firefox
2281	ubuntu	20	0	331964	112016	73332	S	0.6	2.8	1:03.19	Xorg
8251	root	20	0	0	0	0	I	0.6	0.0	0:04.24	kworker/0:1-events
11312	ubuntu	20	0	23616	5816	3640	R	0.6	0.1	0:00.12	top
18	root	20	0	0	0	0	I	0.3	0.0	0:03.01	rcu_preempt
4471	ubuntu	20	0	851860	55048	42664	S	0.3	1.4	0:20.93	gnome-terminal-
1	root	20	0	23520	14832	9712	S	0.0	0.4	0:02.96	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.03	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-netns
12	root	20	0	0	0	0	I	0.0	0.0	0:00.04	kworker/u8:0-ipv6_addrconf
13	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mm_percpu_wq
14	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_kthread
15	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_rude_kthread
16	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_trace_kthread
17	root	20	0	0	0	0	S	0.0	0.0	0:02.41	ksoftirqd/0
19	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_exp_par_gp_kthread_worker/0
20	root	20	0	0	0	0	S	0.0	0.0	0:00.06	rcu_exp_gp_kthread_worker
21	root	rt	0	0	0	0	S	0.0	0.0	0:00.08	migration/0
22	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/0

Q 2. Write a C program using the fork() system call that generates this sequence in the child process. The starting number will be provided from the user. For example, if 8 is passed as a parameter on the command line, the child process will output 8, 4, 2, 1. Because the parent and child processes have their own copies of the data, it will be necessary for the child to output the sequence. Parent invoke the wait() call to wait for the child process to complete before exiting the program. Ensure that a positive integer is passed on the command line.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {

    if (argc != 2) {
        printf("Usage: %s <positive integer>\n", argv[0]);
        return 1;
    }

    int n = atoi(argv[1]);

    if (n <= 0) {
        printf("Please enter a positive integer.\n");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0) {
        printf("Child process sequence: ");

        while (n > 0) {
            printf("%d ", n);
            n = n / 2;
        }

        printf("\n");
        exit(0);
    }

    else {
        wait(NULL);
    }
}
```

```
        printf("Parent: Child completed.\n");  
    }  
  
    return 0;  
}
```

Output:

```
ubuntu@ubuntu:~/test$ gcc fork_sequence.c -o fork_sequence  
ubuntu@ubuntu:~/test$ ./fork_sequence 8  
Child process sequence: 8 4 2 1  
Parent: Child completed.
```

Q 3. Check output and Reason for it.

What is the output of the following program?

```
main()
{
    int a = 10;
    if(fork()) == 0)
        a++;
    printf("%d\n",a);
}
```

Output:

10

11

The following C program is executed on a Unix / Linux system:

```
#include <unistd.h >
int main() {
    int i;
    for (i = 0; i < 10; i++)
        if (i % 2 == 0) fork();
    return 0;
}
```

The total number of child process created is _____ .

Output:

31

Q4. What happens if parent is killed (status of child), what happens when child is killed, what happens when terminal is killed (). Try these in different sequences - parent, terminal; child terminal; terminal parent etc.

Answer:

Case 1: Parent is killed (child still running)

Parent dies first

Child is still alive

Result:

Child becomes an **orphan process**

It is adopted by **init/systemd (PID 1)**

Child continues running normally

Parent PID of child becomes **1**

So:

child runs

no crash

OS takes care of it

Case 2: Child is killed

Child dies first

Parent is still alive

Result:

Child becomes a **zombie** temporarily

Parent must call `wait()` to collect status

After wait zombie disappears

If parent never calls wait:

zombie stays until parent exits

Zombie = dead process entry still in process table

Case 3: Terminal is killed

When terminal closes:

Linux sends **SIGHUP** to processes attached to terminal

Default behavior:

both parent and child terminate

Unless program ignores SIGHUP.

So:

terminal closes
both processes die

Different sequences

Parent → Terminal

Kill parent first:

child becomes orphan
Then close terminal:

child gets SIGHUP
child dies

Final: no process running

Child → Terminal

Kill child:

zombie until parent waits
Then close terminal:

parent dies
zombie cleaned automatically

Final: nothing left

Terminal → Parent

Close terminal first:

both parent and child die immediately
No orphan, no zombie

Q 5. Process Variables and System Calls

a) Create a child process using fork() and explore the effects of:

exec()

wait()

yield()

sleep()

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sched.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child: PID = %d\n", getpid());
        printf("Child sleeping 2 seconds...\n");
        sleep(2);

        printf("Child yielding CPU...\n");
        sched_yield();

        printf("Child executing 'ls' using exec\n");
        execl("/bin/ls", "ls", NULL);

        printf("Exec failed\n");
        exit(1);
    }

    else {
        printf("Parent: PID = %d\n", getpid());
        printf("Parent waiting for child...\n");
        wait(NULL);
        printf("Parent resumes after child finishes\n");
    }
}
```

```
    return 0;
}
```

Output:

```
ubuntu@ubuntu:~$ ./progA
Child: PID = 17982
Parent: PID = 17981
Parent waiting for child...
Child sleeping 2 seconds...
Child yielding CPU...
Child executing 'ls' using exec
Desktop Documents Downloads Music Pictures Public Templates Videos progA progA.c
Parent resumes after child finishes
```

b) Verify whether local, global, and static variables are shared between parent and child processes.

Code:

```
#include <stdio.h>
#include <unistd.h>

int global = 10;
static int s = 20;

int main() {
    int local = 30;

    pid_t pid = fork();

    if (pid == 0) {
        global++;
        s++;
        local++;

        printf("\nChild values:\n");
        printf("global = %d\n", global);
        printf("static = %d\n", s);
        printf("local = %d\n", local);
    }

    else {
        sleep(1);

        printf("\nParent values:\n");
        printf("global = %d\n", global);
        printf("static = %d\n", s);
    }
}
```

```

        printf("local = %d\n", local);
    }

    return 0;
}

```

Output:

```

ubuntu@ubuntu:~/test$ ./progB

Child values:
global = 11
static = 21
local = 31

Parent values:
global = 10
static = 20
local = 30

```

c) Verify the parent of a parent process.

Code:

```

#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Current PID: %d\n", getpid());
    printf("Parent PID : %d\n", getppid());

    return 0;
}

```

Output:

```

ubuntu@ubuntu:~/test$ ./progC
Current PID: 18033
Parent PID: 17912

```